

Institut Supérieur d'Informatique et de Multimédia de Gabes 	Année Universitaire : 2025/2026
	Module : Système d'exploitation 2
	Enseignante : Y.BEN JMAA
	TP2 : Gestion des processus

Objectifs

1. Utilisation de la primitive de base : fork() pour la création d'un processus
2. Compilation et exécution des programmes en langage C

I. Primitive de base

L'appel système **fork()** duplique le processus. Pour savoir dans quel processus (père/fils) on se trouve, il faut regarder la valeur retournée par la fonction :

- ✓ 0 pour le processus fils
- ✓ PID (fils) pour le processus père
- ✓ 1 en cas d'échec

Services POSIX :

- getpid() : retourne l'identificateur du processus courant
- getppid() : retourne l'identificateur du processus père
- sleep(int sec) : le processus appelant s'endort pour sec secondes
- wait(int * status): suspend l'exécution du processus appelant jusqu'à ce que ses processus fils se terminent. Si un processus fils s'est déjà terminé, wait() retourne le résultat immédiatement. Si status n'est pas un pointeur nul, le status du processus fils (valeur retournée par exit()) est mémorisé à l'emplacement pointé par status. wait() retourne le PID du processus fils si le retour est dû à la terminaison du processus fils ; -1 en cas d'erreur
- exit(int status): termine un processus. La valeur status est envoyée au processus père. Par convention en cas de terminaison normale, un processus doit retourner la valeur 0.

Exercice 1

1. Saisir le programme fork1.c ci-dessous :

```
#include<stdio.h>
#include<unistd.h>
int main()
{
    pid_t pid;
    pid=fork();
    printf("hello je programme\n");
    return(0);
}
```

2. Faire la compilation de fork1.c (l'exécutable aura comme nom f1)
3. Lancer l'exécution de f1
4. Expliquer brièvement le résultat de l'exécution

Exercice 2

1. Saisir le **programmefork2.c** ci-dessous :

```
#include<stdio.h>
#include<unistd.h>
#include<stdlib.h>
#define result 7
int main()
{
    pid_t pid;
    switch(pid=fork())
    {
        case -1: { perror("création impossible"); exit(1); }
        case 0: { printf("%d fils de %d \n",getpid(),getppid());
                  exit(result);}
        default: { printf("%d père de %d \n",getpid(),pid); wait() ; }
    }
}
```

1. Faire la compilation de fork2.c (l'exécutable aura comme nom f2)
2. Lancer l'exécution de f2
3. Expliquer brièvement le résultat de l'exécution

Exercice 3

1. Saisir le programme **fork3.c** ci-dessous :

```
#include<stdio.h>
#include<unistd.h>
#include<sys/types.h>
int main()

{

    pid_t pid1,pid2;
    pid1=fork();
    pid2=fork();
    pause();
    return(0);

}
```

2. Faire la compilation de fork3.c (l'exécutable aura comme nom f3)
3. Lancer l'exécution de f3
4. Combien de processus sont lancés à l'exécution de ce programme

Exercice 4

1. Ecrire un programme **fork4.c** qui crée deux fils, l'un affiche les entiers de 1 à 50, l'autre de 51 à 100.
2. Faire la compilation de fork4.c (l'exécutable aura comme nom f4)
3. Lancer l'exécution de f4, que remarquer vous ?
4. Modifier le programme précédent pour que l'affichage soit 1 2 3 ...100 (synchroniser l'affichage)

Exercice 5

```
#include <stdio.h>
#include <unistd.h>
Void main()
{
    Pid_t rf;    /*le résultat de fork*/
    printf("\n  Le père démarre\n");
    rf=fork();  /*Appel système fork*/
    if (rf >0)
    {
        printf("\n Je suis le père \n ");
        printf("\n Je suis en attente de mon fils\n ");
        wait();
        printf("\n Mon fils a termine \n");
    }
    else
    if (rf==0)
    {
        printf("\nJe suis le fils\n Mon PID est: %d", (int)getpid());
        printf("\n Je vais dormir pour 10 secondes \n");
        sleep(10);
    }
    else
    printf("\n Erreur d' execution de fork\n");
}
```

1. Compiler et exécuter le programme ci-dessus. Interpréter le résultat obtenu.
2. Exécuter le programme en arrière plan. Lancer la commande `ps -f`. interpréter le résultat obtenu.
3. Supprimer l'appel système « `wait()` ». Compiler et exécuter de nouveau votre programme. Que remarquez-vous?
4. Lancer le programme en arrière plan. Invoquer la commande `ps -f`. Que remarquez-vous?
5. Ajouter une variable globale à votre programme. Modifier et afficher cette variable dans le code de père, dans le code de fils et à la fin du programme. Interpréter le résultat obtenu.